Alien Coding as an Empirical Program: Replication, Learning Rules, and Self-Improving Sequence Synthesis

Oruži*

Draft, July 2026

Abstract

We develop a reproducible empirical program around "alien coding": learning to propose small programs that reproduce integer sequences, checking those programs independently, and returning verified discoveries to the learner. The program has two deliberately different experimental instruments. A recursive tree neural network coupled to guided tree search provides a compact causal testbed for changing the learning rule while holding the architecture and search fixed. A bidirectional recurrent sequence-to-sequence model with attention is the higher-capacity mainline for studying learned program priors and multi-generation feedback. On the tree testbed, an upstream backpropagation result is replicated, a fixed-step state-based predictive-coding variant reaches 93% of the one-generation backpropagation yield at roughly ten times the training cost, and a deterministic paired three-world campaign tests whether that gap compounds over generations. The common generation-4 snapshot is compounding-shaped, with a paired geometric mean yield ratio of 0.648, but the registered verdict is reserved for generation 5. On the recurrent model, two independently trained historical-port runs add 3,140 and 3,080 checker-verified sequences; a fidelity-corrected reference and discriminating implementation oracles are now available, while its preregistered bounded check remains open after a checker-method substitution and a target-association serialization defect were found. We separate verified discovery, target-conditioned synthesis, and unconditional hindsight coverage; report resource costs in human units; and specify the unrolled-graph oracles required before applying predictive coding to an attention-based recurrent model.

1 The empirical question

An integer-sequence synthesizer receives a finite prefix of a sequence and proposes a program in a small combinator language. This setting follows the "alien coding" program of Gauthier, Olsak, and Urban [1]. A separate evaluator then asks whether the program reproduces a target sequence. The central scientific object is not a single neural model, but the closed loop

$$\text{verified corpus} \longrightarrow \text{learner} \longrightarrow \text{proposal process} \longrightarrow \text{independent checker} \longrightarrow \text{verified corpus}.$$

This loop supports three questions that should not be collapsed into one:

1. Does learning ignite useful program search relative to an untrained or random proposal process?

*Oruži names the AI research collaborators who carried out this work: Anthropic Claude (Opus 4 and Fable 5) and OpenAI Codex (GPT-5.6-Sol) — replication, implementation, experiments, and drafting. Research direction and oversight: Zar Goertzel.

2. Given fixed architecture and search, how does the learning rule change proposal quality, cost, and multi-generation feedback?

3. Does a sequence-to-sequence model use the requested sequence, or does it merely learn a broadly useful unconditional program prior?

The verified language, recognizer, synthesis contracts, leakage-safe corpus view, and graded extensional and intensional measurements are treated in the companion account *OEIS Program Synthesis, Verified Semantics, and Graded Similarity* [3]. That account deliberately contributes a verified and measured object layer rather than another synthesizer. The present paper owns the empirical systems, interventions, longitudinal estimands, and resource measurements. Keeping the two accounts separate prevents results from different architectures and corpora from being mistaken for claims about the verified semantics itself.

1.1 Evidence discipline

Every result below is typed by its evidence:

- A *verified discovery* is a distinct sequence identifier returned by the independent program checker beyond a frozen baseline set. Verification means agreement on the frozen finite 20-term observation; it does not establish identity of the corresponding infinite sequences.
- A *conditional target hit* requires at least one canonical program harvested from a proposal occurrence requested for target T to compute T. A *direct-full hit* uses the program represented by the complete normalized emission, whereas a *harvested-prefix hit* uses a proper harvested prefix. The two target sets can overlap, and their union is the conditional-hit set. The target–proposal–origin association must survive parsing and checking.
- *Hindsight coverage* checks a proposal multiset against all sequences without regard to the target that elicited each proposal. It measures the usefulness of a program prior, not sequence conditioning.
- A replicated claim requires at least two independent training seeds. A paired longitudinal claim treats independent seed-worlds, not programs or search nodes, as the inferential units.

Interpretations and stopping rules are fixed before their endpoint is observed. Inputs, source, configuration, logs, outputs, and completion records are content-manifested. Retries create new attempts rather than overwriting evidence. These rules matter because program checking and search are long-running stateful computations: a partial check must never become a zero, and an operational restart must never silently become a new scientific condition.

2 A tree-neural causal testbed

The first instrument is the recursive tree neural network and guided search used by the QSynt line of work [2]. Each operator owns a shared affine map. A program DAG is evaluated bottom-up to produce node embeddings; policy heads predict search actions. There are no value heads in the configurations studied here, so the learned policy prior is the only neural guidance supplied to search.

2.1 Upstream replication and one-generation ignition

The upstream pipeline was rebuilt with its Standard ML search, C/BLAS tree learner, and HOL-based checker. At the small replication setting, a random generation-0 search found 861 sequences and one 100-epoch backpropagation training round followed by guided search raised the cumulative count to 4,956, a 5.76× ignition. A second seed produced 863 → 4,772. Thus the two-seed mean guided yield is 4,864, with the two runs within 1.9% of that mean. These runs establish an operational and scientific baseline; they are not compared numerically with the recurrent model, whose language, initial corpus, proposal budget, and checker path differ.

2.2 Exact learning-rule intervention

For a hidden node n, let z_n be its post-activation latent and let $f_n(z_{\mathrm{ch}(n)}; \theta_{o(n)})$ be the prediction made by the matrix shared by operator $o(n)$. For a policy head h, let $g_h(z; \theta_h)$ be its tanh output and t_h its soft target. The implemented predictive-coding energy is represented by

$$F(z, \theta) = \frac{1}{2} \sum_h \|t_h - g_h(z; \theta_h)\|_2^2 + \frac{\beta}{2} \sum_{n \in \mathcal{H}} \|z_n - f_n(z_{\mathrm{ch}(n)}; \theta_{o(n)})\|_2^2.$$

This is a finite-step, digitally simulated instance of state-based predictive coding (sPC): the inferred variables are node states, and the inference signal moves through local residual factors. All tree results in this paper are claims about this exact sPC implementation, not about predictive coding as a whole. Recent approaches attack deep sPC's digital scaling problem by changing initialization and residual scaling (μPC) [6] or by reparameterizing inference in terms of prediction errors (ePC) [7]; neither method is part of the tree campaign. Latents are initialized to the ordinary forward activations. With weights fixed, the implementation performs K simultaneous latent-gradient steps

$$z_n \leftarrow z_n - \eta_z \nabla_{z_n} F.$$

It then performs one local outer-product update per node. The selected arm, denoted PC32, fixes $K = 32$, $\eta_z = 0.1$, and $\beta = 1$. The BP arm uses the existing reverse pass. Both arms use batch-one stochastic updates and 100 epochs. The exact outer learning-rate schedule is 8, 4, 2, and 10^{-4} on epoch blocks 0–24, 25–49, 50–74, and 75–99, respectively. Each parameter update is clipped to $[-4, 4]$; latent gradients are not clipped.

The label *state-based predictive-coding variant* is important. Settling itself holds all weights fixed. After settling, however, the implementation visits node occurrences in reverse order, recomputes each local residual from the live shared operator matrix, and immediately mutates that matrix. If an operator occurs twice, the later occurrence observes the first occurrence's update. BP computes its reverse signals before its update loop, so its per-occurrence contributions are based on fixed forward values. The registered campaign therefore tests this exact order-dependent PC32 implementation. A future exact-schedule control must accumulate all tied-operator contributions under one fixed parameter state and apply them once.

The PC path was checked against independent numerical goldens on a two-node case and on a shared multi-parent DAG; deliberately broken variants fail those comparisons. The BP path remains a separately selected branch. These implementation oracles establish agreement with the specified local update, not equivalence to BP.

2.3 Settling-depth pilot and the decisiveness hypothesis

Table 1 varies only the number of PC latent-settling steps. BP, $K = 4$, and $K = 32$ have two seeds; $K = 8$, $K = 16$, and $K = 64$ have one each. Yield and training cost are therefore two-seed means

only for BP, $K = 4$, and $K = 32$. The RMS entries for those three settings are likewise two-seed means; the other RMS entries are single runs. Entropy is measured from one model per setting and is the mean entropy of that model's legal-action prior sampled during search. Training cost is relative to the two-seed BP mean at the same small setting.

Table 1: One-generation settling-depth pilot. $K = 8$, $K = 16$, and $K = 64$ are single-seed results; entropy uses one model at every setting.

learner	guided yield	BP yield fraction	train cost	policy RMS	prior entropy (nats)
PC, $K = 4$	3,052	0.63	2.2×	0.097	1.479
PC, $K = 8$	3,281	0.67	3.5×	0.083	1.230
PC, $K = 16$	4,037	0.83	6.0×	0.072	1.149
PC, $K = 32$	4,527.5	0.93	9.9×	0.069	0.975
PC, $K = 64$	4,189	0.86	19.3×	0.042375	1.153861
BP	4,864	1.00	1.0×	0.137	0.671

The learner prints a per-head root-mean-square policy error and averages those roots; the column is therefore not a mean-squared error despite the historical log label. Deeper settling monotonically improves PC's policy fit and downstream yield through $K = 32$. At $K = 64$, policy RMS improves again, but entropy rises from 0.975 to 1.153861 and yield falls from 4,527.5 to 4,189; its 1,885.24-second training time is about 19.3× the BP mean. Thus fit continues to improve after entropy and yield turn over, although the $K = 64$ observation has only one seed. PC32's two yields (4,744 and 4,311) are both below the two BP yields (4,956 and 4,772). Lower trainer-reported RMS error on the soft policy targets therefore does not imply better budgeted search.

Prior entropy supplies a concrete mechanism hypothesis. The BP prior is more decisive; deeper PC settling sharpens the PC prior in the same direction as its improving search yield. The observations are not independent at the level of millions of node visits, and the single-seed learners have only one seed. Entropy is consequently a correlated diagnostic, not an established cause. A causal follow-up should alter decoder/search temperature on frozen checkpoints and measure target rank, margin, calibration, invalid-program rate, unique proposals, and verified coverage. Lower entropy accompanied by worse rank or coverage is not stabilization.

The frozen 4,096-state generation-1 panel—2,048 train-source states plus 2,048 historical-validation states—provides a stronger paired diagnostic. In all three worlds PC32 has higher entropy, lower top-1 accuracy, and worse target rank than BP (Table 2). W3 simultaneously has the least diffuse PC32 prior and the best PC32/BP coverage ratio, an exploratory alignment rather than a causal result.

Table 2: Paired generation-1 common-panel diagnostics and guided-search coverage.

world	arm	entropy	top-1	mean target rank	coverage
W1	BP	0.475	0.545	2.557	8,752
W1	PC32	1.253	0.385	3.006	6,465
W2	BP	0.475	0.524	2.619	8,361
W2	PC32	1.298	0.392	2.957	6,093
W3	BP	0.485	0.543	2.646	8,627
W3	PC32	1.196	0.387	2.975	7,548

2.4 Deterministic paired multi-generation campaign

The pilot runs used clock-seeded search and training shuffles, so they could not answer a multi-generation causal question. The paired campaign replaces every such draw with a recorded deterministic stream:

- Three independent worlds use master seeds 26,071,001–26,071,003.
- A domain-separated SHA-256 derivation supplies all controller, target-job, weight, and epoch-order seeds. Every derived seed lies in the accepted 31-bit range.
- Within a world, BP and PC32 begin from byte-identical generation-0 corpora, model parameters, and epoch order. Their 50 search jobs use identical $(\text{job}, \text{target}, \text{seed})$ ledgers and reset search caches per job.
- After generation 1, each arm trains on its own cumulative verified corpus. The keyed sampling rule is shared, but byte-identical batches would be impossible and scientifically wrong once the corpora diverge.
- BP-first and PC-first order alternates, and only one trajectory runs at a time. Each phase writes a manifest and an independently checkable completion record in a fresh attempt root.

The medium setting uses 128-dimensional node states, four search workers, 50 targets per generation, and a nominal 200 target-seconds per target: nominally 10,000 target-seconds, or 2.78 target-hours, per search arm. One job overshot to 247.46 seconds, so 10,000 is a target budget rather than an exact realized total. Successful search wall time is 43.52–44.54 minutes because the four jobs run in parallel with orchestration overhead.

Let $C_{s,a,g}$ be the cumulative number of distinct verified sequences in world s, arm a, and guided generation g. The primary paired ratio and change are

$$R_{s,g} = \frac{C_{s,\mathrm{PC32},g}}{C_{s,\mathrm{BP},g}}, \qquad T_{s,G} = R_{s,G} - R_{s,1}.$$

For each world we also fit the ordinary least-squares slope of $R_{s,g}$ over $g = 1, \ldots, G$. The endpoint summary is the geometric mean of the three $R_{s,G}$ values with a two-sided 90% t interval on log ratios, treating worlds as the three inferential units.

At $G = 5$, the frozen classifier uses the following precedence order; the first satisfied label is reported:

- *compounds*: median $T_{s,5} \le -0.03$ and all three slopes are non-positive;
- *closes to parity*: all slopes are non-negative and the endpoint interval lies in $[0.97, 1.03]$;
- *closes materially*: median $T_{s,5} \ge 0.03$ and all slopes are non-negative;
- *persists*: median $T_{s,5}$ lies within ± 0.03 and parity equivalence is not established;
- *mixed/inconclusive*: none of the above.

These labels describe the total effect of a learning rule inside its own feedback loop. They do not isolate a same-corpus gradient effect after generation 1.

2.5 Interim trajectory

Table 3 reports the common completed generation-4 snapshot audited before the generation-5 endpoint; all three worlds are verified through generation 4. Parentheses contain each arm's own novel increment.

Table 3: Cumulative verified sequence counts in the paired medium campaign.

world	generation	BP C (increment)	PC32 C (increment)	$R = \text{PC32/BP}$
W1	0	1,623	1,623	—
W1	1	8,752 (7,129)	6,465 (4,842)	0.738688
W1	2	11,709 (2,957)	6,914 (449)	0.590486
W1	3	13,224 (1,515)	9,067 (2,153)	0.685647
W1	4	14,566 (1,342)	9,486 (419)	0.651243
W2	0	1,587	1,587	—
W2	1	8,361 (6,774)	6,093 (4,506)	0.728741
W2	2	11,291 (2,930)	8,031 (1,938)	0.711274
W2	3	13,250 (1,959)	8,359 (328)	0.630868
W2	4	15,054 (1,804)	9,195 (836)	0.610801
W3	0	1,640	1,640	—
W3	1	8,627 (6,987)	7,548 (5,908)	0.874928
W3	2	11,510 (2,883)	7,548 (0)	0.655778
W3	3	13,267 (1,757)	8,820 (1,272)	0.664807
W3	4	14,793 (1,526)	10,111 (1,291)	0.683499

At the common generation-4 endpoint, the ratios are 0.651243, 0.610801, and 0.683499. Their geometric mean is 0.647829 and the descriptive 90% endpoint interval under the registered estimator is $[0.589059, 0.712462]$. Changes from generation 1 are -0.087446, -0.117939, and -0.191429; slopes through generation 4 are -0.016718, -0.043422, and -0.056526. BP exceeds PC32 in every guided generation 1–4; generation 0 is equal by construction. This is directional replication and is *compounding-shaped*, but generation 4 is not a registered interpretation endpoint and this is not the generation-5 verdict.

W3's zero PC increment at generation 2 is a verified zero, not a missing phase: all jobs and target-seconds completed and the loadable PC model was used. PC then recovered 1,272 arm-own discoveries at generation 3. Set analysis shows that many apparent rebounds are delayed rediscoveries of BP contributions. For example, only 286 of W1 PC's 2,153 generation-3 additions were new to the prior paired union. The corresponding new-to-union counts for W1 generations 2–4 are 165, 286, and 34; for W2 they are 992, 104, and 82; for W3 generations 2–4 they are 0, 214, and 48. PC32 is therefore burstier and more path-dependent at this snapshot, while BP contributes more smoothly to the paired frontier.

2.6 Resource cost

Search is the stable part of the budget: each medium arm takes 43.52–44.54 minutes and peaks between 6.25 and 9.57 GiB of resident host memory. Through the common generation-4 endpoint, the twelve BP training phases use 7.26 hours and the twelve PC32 phases use 42.06 hours. Search adds 8.73 hours per arm, making core active train-plus-search time 15.99 hours for BP and 50.80 hours for PC32. Matched world/generation per-example PC/BP training ratios range from 8.01 to

10.47, with a median of 9.53. Peak training memory is lower than search memory (about 3.1 GiB for PC and 4.1 GiB for BP); serial training time, not memory, is the dominant PC cost. These successful-phase totals exclude failed attempts, preflight work, diagnostic panels, launch-gate waits, and inter-phase waits.

Table 4: Observed training wall time in minutes. Generation denotes the guided search enabled by training the preceding cumulative corpus.

world	arm	g1	g2	g3	g4
W1	BP	3.64	29.45	46.77	60.41
W1	PC32	37.71	207.76	231.59	356.66
W2	BP	3.20	28.24	48.96	67.06
W2	PC32	33.47	189.56	282.11	306.43
W3	BP	3.28	28.94	47.53	68.08
W3	PC32	34.31	256.92	259.62	327.78

2.7 Registered generation-5 endpoint

Generation-5 result: ***pending verified endpoint.*** The verified endpoint will report all six cumulative and own-novel counts; each world's $R_{s,5}$, $T_{s,5}$, and slope; the geometric mean and registered 90% interval; new-to-paired-union contributions; common-panel diagnostics; and measured resource totals. The interpretation will be selected mechanically from the labels in Section 2.4. No generation-6 run is part of this endpoint.

The older saturation rule is first eligible at generation 7 because it requires two successive increments below 0.5% in every trajectory. It is therefore a separate continue-to-generation-7 fork, not an automatic consequence of the generation-5 analysis.

3 The recurrent sequence-to-sequence mainline

The mainline architecture maps a tokenized sequence prefix to a program token sequence. It has a one-layer bidirectional LSTM encoder, two 512-unit decoder LSTM layers with input feeding, and scaled Luong attention. Beam search proposes programs, while the same independent evaluator used by the upstream synthesis system determines which sequences they compute. Unlike the tree testbed, this model can learn both a conditional mapping from input sequences and an unconditional prior over syntactically useful programs.

3.1 Historical PyTorch port: real effect, bounded interpretation

The first modern port, retained as version 0, trained two seeds for 12,000 SGD steps on the 3,771-sequence base corpus. Full decodes followed by whole-corpus program checking yielded 3,140 and 3,080 new distinct sequences. The two discovery sets overlap on 2,527 sequences and add 3,693 distinct sequences in union. These counts have been recomputed from the checker outputs and set differences.

An untrained network produced zero new sequences on a complete post-hoc assay of 2,000 targets at beam 240, compared with 746 and 780 for the two trained models under the same non-harvesting check. This is strong two-seed evidence that training learns program syntax and a useful proposal distribution. It is not closure of the original full-target, prefix-harvesting control, which

did not complete; and no randomized target-association control was run. The supported claim is a replicated learned-program-prior effect, not yet a sequence-conditioning claim.

Source audit also found that version 0 was not a faithful reference implementation. It applied the forget bias to both PyTorch LSTM bias vectors, giving an initial effective offset of two and 8,192 extra trainable bias parameters; shared encoder input dropout between directions; omitted dropout from the previous-attention and upper-decoder inputs; and decoded to length 32 rather than the historical horizon of 280. The version-0 evidence remains valid for that exact model but is not used as the BP reference for a future PC comparison.

3.2 Fidelity-corrected version 1

Version 1 uses a single effective trainable LSTM bias, initializes all gate biases to zero, adds the forget offset once, and freezes the duplicate recurrent bias at zero. Its trainable parameter count is 10,781,697. It applies independent input dropout to the two encoder directions, to the complete decoder input-feeding vector, and to the second decoder layer's input. The decode horizon is 280 and the beam is 240.

Five retained oracles test overfitting 32 examples, beam-score agreement, attention masking, EOS termination, and deterministic logits. Each has a positive case and a deliberately broken negative case; all transcripts are bound by one SHA-256 manifest. Two version-1 seeds completed the same 12,000-step recipe in 17.4 and 16.5 minutes. On a frozen 21,061-target grid, their beam decodes took 168 and 162 seconds; the untrained decode took 1,058 seconds. The calibrated decode peak was 20.3 GiB of GPU memory.

The complete non-harvesting check produced the provisional counts in Table 5. They strongly suggest replicated ignition, but they do not close the preregistration.

Table 5: Version-1 bounded-grid check, pending provenance-correct recheck. "New" is beyond the 3,771-sequence baseline; grid coverage is hindsight coverage.

arm	unique programs checked	new sequences	grid targets covered
trained seed 1	1,248,629	1,744	318 / 21,061
trained seed 2	1,227,124	1,630	318 / 21,061
untrained	1,668,705	0	3 / 21,061

Two audit findings prevent a stronger statement. First, the preregistration fixed a prefix-harvesting checker inside immutable content-addressed attempts, whereas the completed run substituted a non-harvesting checker, reused mutable work roots, and wrote no completion record. The three arms were checked consistently, but under a different method and weaker provenance discipline. Second, the target-association analysis serialized a set of multi-token canonical programs by joining them with spaces and reconstructed the set by splitting on spaces. It therefore compared complete sidecar programs against individual tokens. The reported conditional, direct-full, harvested-prefix, and randomized-association numbers are invalid. The registered target-conditioning endpoint remains unresolved.

The content-addressed broker in Section 3.3 is consequently a scientific requirement, not merely operational cleanup. Its implementation and lightweight unit suite exist, but the broker remains under validation until a live differential oracle agrees with the stock HOL checker and the complete prefix-harvesting N4 recheck finishes successfully. Until that recheck is complete, version 1 is a fidelity-corrected implementation reference with provisional ignition evidence.

3.3 Content-addressed proposal and check broker

The broker keeps three identities separate. A *proposal-occurrence ID* binds the model and decode identities to the sidecar source-line number and raw-row hash. A distinct *normalized-emission ID* identifies the whitespace-normalized reverse-order string, independent of how many proposal occurrences emitted it. Prefix harvesting then produces one or more *canonical-program IDs*, each derived from an unambiguous canonical postfix program. Explicit relations connect proposal occurrences to emissions, emissions to harvested programs and origin kinds, and programs to checker coverage. Programs are never serialized by an ambiguous token delimiter.

A broker request hashes the canonical candidates, target and sidecar slices, model, checker source and configuration, environment, and baseline solution set. Each request has immutable numbered attempts, and each chunk has its own isolated checker root. Completion requires all expected chunks, their output hashes, and aggregate invariants. Only then is a final completion object written and its digest stored in the attempt's `DONE` record. A retry creates a new attempt and may adopt a prior chunk only after validating its request and completion hashes. No checker configuration or experiment directory is edited in place.

This representation makes four related measurements explicit. Writing $p \rightsquigarrow q$ when canonical program q was harvested from proposal occurrence p, and $\mathrm{full}(p, q)$ when it represents the complete normalized emission,

$$\begin{aligned}
\text{conditional hits} &= \big|\{T : \exists p, q,\ \mathrm{requested}(p) = T \wedge p \rightsquigarrow q \wedge \mathrm{computes}(q, T)\}\big|,\\
\text{direct-full hits} &= \big|\{T : \exists p, q,\ \mathrm{requested}(p) = T \wedge p \rightsquigarrow q \wedge \mathrm{full}(p, q) \wedge \mathrm{computes}(q, T)\}\big|,\\
\text{harvested-prefix hits} &= \big|\{T : \exists p, q,\ \mathrm{requested}(p) = T \wedge p \rightsquigarrow q \wedge \neg\,\mathrm{full}(p, q) \wedge \mathrm{computes}(q, T)\}\big|,\\
\text{hindsight coverage} &= \big|\{T : \exists q,\ \mathrm{computes}(q, T)\}\big|.
\end{aligned}$$

The direct-full and harvested-prefix target sets may overlap; their union is exactly the conditional-hit set. A keyed permutation of the requested relation supplies C2 without changing the proposal multiset or hindsight coverage, and C2 reports the same conditional, direct-full, and harvested-prefix sets together with their overlap.

3.4 Generation-2 loop gain and multi-generation PC–BP

The first prospective version-1 longitudinal experiment separates loop gain from learning rule comparison. Independent worlds begin from the same base corpus. Within each world, a feedback arm adds only broker-verified generation-1 discoveries to a canonically ordered cumulative ledger; a no-feedback arm retrains from the unchanged base corpus. The pair shares its generation-2 initialization, optimizer-update budget, batching rule, target grid, beam, and checker. Actual example and token exposures are reported because equal optimizer steps are not equal data exposure when corpora differ.

The preserved version-0 generation-2 decodes are useful only as a broker load test. Their feedback corpus pooled worlds, changed multiplicities, omitted rows without a manifested rule, and exposed the treatment to 5.29% more examples under the nominally equal update count. A no-new-training-or-decode-cost broker recheck can be reported as a descriptive pilot, but checking still consumes compute and cannot repair that causal design after the fact.

Version-1 generation-2 loop-gain result: ***pending verified endpoint.*** The endpoint will report paired new verified sequences beyond each world's generation-1 frontier; conditional, direct-full, harvested-prefix, and hindsight hits; the overlap of the two origin-specific target sets; corpus composition and exposure; proposal validity and diversity; and train/decode/check cost for at least two independent worlds.

Only after the recurrent-PC gates in Section 4 pass does the mainline compare BP and PC inside the NMT loop. That experiment uses three independent worlds, shared generation-0 state, arm-own cumulative corpora thereafter, a mandatory generation-3 endpoint, and a prospectively frozen neutral feasibility rule for any extension to generation 5. Equal optimizer updates and batch-size policy define the primary estimand; matched wall time is a separate secondary question.

4 Predictive coding on an attention-based recurrent model

Applying the tree PC update directly to a bidirectional LSTM would be neither a faithful recurrent implementation nor a fair BP comparison. Bidirectionality, LSTM multiplicative gates, a loss at every decoder step, attention branches, input feeding, masking, unequal paths, and parameters tied over time all matter.

4.1 Explicit unrolled computation graph

The first implementation exposes latent nodes for encoder forward and backward (h, c) at each source position; both decoder layers' (h, c) at each target position; attention scores, normalized attention, context, and attentional output; and token logits. Source tokens and teacher-forced previous target tokens are fixed. Padding and post-EOS positions are masked exactly as in BP. A coarse LSTM transition is initially one differentiable factor rather than a collection of gate-level latents.

Let z_t^ℓ be the explicit token-logit latent and let f_t^ℓ predict it from the attentional state. For the other latent groups g, the proposed energy is

$$F(z,\theta) = \sum_{t\in\mathcal{M}} \mathrm{CE}\Big(y_t, \mathrm{softmax}(z_t^\ell)\Big) + \frac{\lambda_\ell}{2}\sum_{t\in\mathcal{M}} \|z_t^\ell - f_t^\ell(\mathrm{pa}(z_t^\ell);\theta)\|_2^2 + \frac{1}{2}\sum_{g\neq\ell}\lambda_g \|z_g - f_g(\mathrm{pa}(z_g);\theta)\|_2^2,$$

where $\mathcal{M}$ is the unmasked token set. Thus cross-entropy consumes the logit latent, while a separate residual factor connects that latent to its predicted logits. Local automatic differentiation may provide vector–Jacobian products for individual factors, but it must not differentiate through the K inference iterations. At each settling step the latent leaves are detached and rebuilt; weights remain fixed. After settling, all contributions to recurrent, attention, embedding, and output parameters are accumulated over times, directions, layers, and tokens before one update is applied.

4.2 Candidate inference parameterizations

The energy and unrolled graph are the scientific object; the numerical method used to infer their internal variables is a separate intervention. The direct continuation from the tree experiment is state-based inference, which optimizes the latents z_g. ePC instead introduces an additive error for each predicted latent,

$$\epsilon_g = z_g - f_g(\mathrm{pa}(z_g);\theta), \qquad z_g = f_g(\mathrm{pa}(z_g);\theta) + \epsilon_g,$$

and reconstructs the graph in topological order. It then uses reverse-mode automatic differentiation through that reconstructed graph to optimize all errors at once, while retaining the local, post-inference weight rule. For a feed-forward hierarchy this is a bijection that preserves the energy and its set of critical points, although state-based and error-based optimization follow different trajectories and can in principle select different minima [7].

This makes ePC a relevant candidate for the depth-through-time problem, not an automatic choice. Its published evidence covers feed-forward MLP, VGG, and ResNet image classifiers; the released core implementation is a sequential vision stack, not an attention-based recurrent model. Likewise, μPC demonstrates that residual parameterization can train very deep sPC classifiers [6], but does not establish a recurrent attention algorithm. We therefore compare state-based and error-based inference on the same unrolled NMT energy before choosing a main PC arm. Error-based inference qualifies only if it preserves the graph and local-weight-update oracles, learns the 32-example problem in at least two seeds, produces a demonstrably non-BP terminal update rather than the one-step BP endpoint, and has a measured cost compatible with the paired experiment. State-based inference remains the direct tree-comparison arm and the diagnostic for finite-settling effects.

4.3 P0: graph-fidelity oracle

P0 is an oracle suite, not a training result. On short padded and unpadded examples it must establish:

1. the unrolled forward logits, hidden and cell states, attention weights, contexts, and loss equal those of the fused version-1 BP model within a frozen tolerance;
2. masked source positions receive zero attention and masked target positions contribute neither loss nor parameter update;
3. repeated-time and repeated-token parameter contributions are summed under one frozen parameter state;
4. the energy is finite and one sufficiently small latent step decreases it;
5. deliberately broken direction order, cell-state recurrence, attention mask, input-feeding edge, EOS mask, and tied-gradient accumulation each fail a named negative test;
6. state and error parameterizations give equal energy on mapped configurations, and zero error reconstructs the fused forward pass.

P1 then compares one update against a numerical gradient, checks long-horizon state/error equilibrium agreement on small tractable graphs, verifies that one-step ePC gives its predicted scaled-BP endpoint, and requires the chosen finite-settling update to separate measurably from BP before it can be called a PC treatment. It also runs a short exact-schedule control. Published Z-IL exactness extends to a considered many-to-one recurrent network [5]; it does not by itself establish exactness for this variable-length, many-output biLSTM encoder–decoder with attention. General computation-graph predictive coding is relevant [4], but leveling, event timing, masking, and tied accumulation remain implementation obligations here.

Explicit unrolling forfeits the fused recurrent kernel and multiplies activation, error, and local-autograd storage. Before P4, 100-step calibrations at several batch sizes and inference budgets must measure actual allocated and reserved GPU memory and wall time. For three worlds through generation 3, the current state-based planning envelope is 35–75 serial GPU-hours for a K8 survivor and 87–231 hours for K32, plus contingency. No ePC envelope is asserted before measuring the recurrent implementation. These ranges are why the gates are scientific stopping points rather than formalities.

Recurrent-PC P0–P4 results: ***pending verified endpoint.*** No NMT PC result is claimed in this draft. A full multi-generation comparison is licensed only by a two-seed P3 pass followed by a preregistered P4 result.

Table 6: Staged recurrent-PC gates. Runtime envelopes after P1 are planning estimates, not measurements.

gate	required evidence	planning envelope
P0	Forward graph fidelity, masks, mapped state/error energy, positive and negative oracles	1–2 focused days
P1	Numerical updates, state/error equilibrium check, BP endpoint and non-BP finite-settling check, tied-weight sum, short exact schedule	3–5 focused days
P2	Overfit 32 examples under at least two seeds for each qualified solver; inference-budget screen; no divergence	1–3 days
P3	2,000 BP/PC steps with identical input state and a disjoint 2,000-target check, at least two seeds	under 3 GPU-hours if K8 envelope holds
P4	12,000-step generation 1, full decode/check, at least two paired seeds	K8: 2–6 GPU-hours per seed; K32: 8–24

5 Stabilization experiments

Four candidate interventions answer different questions and should remain separately named.

1. **Proposal temperature** is the cheapest causal probe of the decisiveness hypothesis. The tree search already renormalizes a powered prior. The recurrent beam currently uses raw log-softmax scores with no temperature. A disjoint target screen should freeze one temperature before a full replay. In the NMT, temperature can change EOS timing, sequence ranking, and beam membership even when the within-prefix token order is unchanged.

2. **Adaptive settling** should stop on normalized energy decrease and latent gradient norm, with minimum and maximum iteration counts and a divergence guard. Output entropy is an outcome diagnostic, not the Lyapunov function and not a valid stopping quantity.

3. **Exact scheduling** is a correctness control. If it reproduces BP exactly, it need not receive its own multi-generation campaign. Its purpose is to validate graph construction, event levels, and tied-gradient accumulation.

4. **Precision-weighted residuals** should begin with fixed, shrunk group precisions. In the NMT these groups include encoder hidden/cell transitions, decoder hidden/cell transitions, attention/context, and the categorical output. Learned precisions require their normalization term and constraints that prevent a trivial collapse.

These experiments distinguish stabilization of inference from calibration of the proposal process. A temperature intervention can improve search while leaving learning untouched; an energy stop can reduce learning cost without changing the decoder; and an exact schedule can validate implementation while offering no performance gain.

6 What can and cannot be compared

The tree and recurrent systems are complementary, not two rows in an architecture leaderboard. The tree testbed begins from roughly 1,600 medium-run discoveries, uses a larger active operator

profile, trains a 128-dimensional recursive model, and allocates budgeted MCTS-like target searches. The recurrent model begins from 3,771 baseline sequences, emits the 14-operator target language, and uses beam search over every target or a frozen target grid. Absolute solution counts confound language, starting corpus, proposal count, evaluator budget, and hardware.

The valid cross-system comparison is methodological:

- the tree instrument isolates an exact learning-rule implementation and reveals how its proposal prior interacts with guided search;
- the recurrent instrument tests whether a larger learned prior ignites, conditions on inputs, and benefits from verified feedback;
- both require independent checking, target-aware identity, paired seed-worlds, and honest separation of discovery, conditioning, and compute.

An architecture comparison would require a separate frozen common language, baseline corpus, target set, checker, and declared proposal/evaluator budgets. Native wall-time curves should still be reported separately because tree search and beam search are intentionally different algorithms.

7 Limitations and current status

The tree multi-generation campaign has three inferential worlds at one medium configuration. Its registered estimand is a total feedback-loop effect after generation 1, not a same-corpus gradient comparison. PC32's post-settle tied-operator update is sequential and order-dependent. The entropy evidence is observational at the checkpoint level, and two settling-depth midpoints have one seed. The generation-5 endpoint is not yet present in this draft.

The recurrent version-0 effect is real for its exact implementation but does not establish historical topology fidelity or sequence conditioning. Version 1 corrects the known architecture deviations and has retained discriminating oracles, but its first bounded check departed from the registered method and provenance protocol. Its initial C2 implementation cannot support a conditioning claim. No version-1 feedback result and no recurrent-PC result are yet claimed.

The most important positive conclusion is therefore also a constraint on interpretation. Learning reliably produces useful program priors in both instruments. The tree evidence shows that lower training error can coexist with a worse search prior, and that sufficient PC settling closes much but not all of the one-generation gap at substantial cost. Whether that gap compounds is a generation-5 question whose verification is still running. Whether feedback and recurrent PC survive in the NMT is a prospective program whose checker broker and graph oracles are part of the scientific method, not post-processing details.

A Artifact-level reporting contract

The final empirical release should generate tables from completion objects rather than from hand-copied logs. Each reported arm should expose at least:

- content hashes for source, configuration, environment, baseline, target ledger, model, candidates, sidecar, checker chunks, aggregate outputs, and completion;
- training examples and tokens, optimizer updates, train/decode/check wall time, peak host and GPU memory, and retained storage;

- cumulative verified sequences, arm-own novelty, novelty beyond the prior paired union, conditional target hits, direct-full hits, harvested-prefix hits, hindsight hits, invalid and valid-prefix fractions, proposal occurrences, normalized emissions, canonical programs, and program/sequence length summaries;
- an explicit status for every expected phase: verified success, verified failure, incomplete attempt, superseded attempt, or absent. Missing and incomplete phases are never numerical zeros.

For NMT candidates, the canonical ledger retains multiplicity separately from identity and records sequence ID, canonical program, source snapshot, world, arm, generation, model, requested target, beam rank and score, direct-full versus harvested-prefix origin, checker identity, and verification attempt. This is sufficient to reproduce cumulative corpora and to audit every distinction used in the paper.

References

[1] T. Gauthier, M. Olšák, and J. Urban. *Alien Coding*. arXiv:2301.11479, 2023.

[2] T. Gauthier and J. Urban. *Learning Program Synthesis for Integer Sequences from Scratch*. AAAI 2023; arXiv:2202.11908.

[3] Z. J. Goertzel. *OEIS Program Synthesis, Verified Semantics, and Graded Similarity: A Unified Account*. Manuscript, 2026.

[4] B. Millidge, A. Tschantz, and C. L. Buckley. *Predictive Coding Approximates Backprop along Arbitrary Computation Graphs*. arXiv:2006.04182, 2020.

[5] T. Salvatori, Y. Song, T. Lukasiewicz, R. Bogacz, and Z. Xu. *Predictive Coding Can Do Exact Backpropagation on Convolutional and Recurrent Neural Networks*. arXiv:2103.03725, 2021.

[6] F. Innocenti, E. M. Achour, and C. Buckley. *μPC: Scaling Predictive Coding to 100+ Layer Networks*. Advances in Neural Information Processing Systems 39, 2025.

[7] C. Goemaere, G. Oliviers, R. Bogacz, and T. Demeester. *ePC: Fast and Deep Predictive Coding in Digital Simulation*. Proceedings of the 43rd International Conference on Machine Learning, 2026; arXiv:2505.20137.